



UNIDAD III. Estándares para el desarrollo web	ACTIVIDAD: DSW-A09. Página HTML, CSS y JavaScript – Centro de Cómputo FEI
TIPO: Actividad individual	

OBJETIVO:

El estudiante incorporará interactividad a una página web institucional previamente desarrollada con HTML y CSS, mediante el uso de JavaScript, el modelo de DOM y la manipulación de documentos, aplicando operaciones de cambio de texto, cambio de estilos, modificación de atributos, creación, inserción y eliminación de elementos, asociación de eventos, uso de funciones flecha, validación de formularios, carga de archivos con arrastrar y soltar, y manejo básico de errores con try/catch.

DESCRIPCIÓN:

Los estudiantes retomarán la página web del Portal Institucional del Centro de Cómputo FEI, desarrollada en la actividad DSW-A07. Página HTML responsivo con CSS- Centro de Cómputo FEI con HTML semántico, CSS Grid y diseño responsivo. A partir de esta base, incorporarán funcionalidades interactivas con JavaScript para reforzar de manera práctica los temas revisados en clase sobre el modelo de DOM, manipulación de documentos y lenguajes de scripting.

Durante la actividad se implementarán diversas mejoras en la página, entre ellas: un panel de mensajes dinámicos, una sección de acciones rápidas, un carrusel de imágenes, validación del formulario de contacto, una zona de carga de archivos con drag and drop, un modo nocturno, así como el uso de funciones flecha y estructuras try/catch en situaciones específicas de interacción y validación.

La finalidad es que el estudiante comprenda que una página web moderna no solo organiza y presenta información, sino que también responde a eventos del usuario, modifica dinámicamente su contenido y mejora la experiencia de interacción mediante scripts del lado del cliente.

MATERIAL Y EQUIPO REQUERIDOS:

- a) Visual Studio Code o cualquier otro editor de código.
- b) Navegador web (Chrome o Firefox).
- c) Carpeta del proyecto desarrollada en la actividad DSW-A07.
- d) Archivo index.html.
- e) Archivo css/estilos.css.
- f) Carpeta img con imágenes del portal.
- g) Crear una carpeta js para almacenar el archivo script.js.



PROCEDIMIENTO:

Paso 1 – Preparar el archivo JavaScript

Dentro de la carpeta del proyecto, crea una nueva carpeta llamada js y dentro de ella el archivo:

js/script.js

Abre el archivo index.html y antes de cerrar la etiqueta </head>, agrega la referencia al archivo JavaScript:

```
<script src="js/script.js" defer></script>
```

Paso 2 – Crear un panel de mensajes dinámicos

Para mostrar mensajes al usuario dentro de la misma página, agregar en index.html, inmediatamente después de la etiqueta de apertura de <main id="contenido">, el siguiente bloque:

```
<section id="panel-mensajes" aria-label="Mensajes dinámicos del portal">  
  <h2>Estado del portal</h2>  
  <p id="mensaje-dinamico">Bienvenido al Portal Institucional del  
Centro de Cómputo.</p>  
</section>
```

Paso 3 – Cambiar texto del documento

Abre js/script.js y agregar el siguiente código:

```
//Cambiar texto del panel de mensajes dinámicos  
const mensajeDinamico = document.getElementById("mensaje-dinamico");  
mensajeDinamico.textContent = "Portal cargado correctamente. JavaScript se  
encuentra activo.";
```

La instrucción getElementById() localiza el elemento por su identificador y textContent reemplaza su contenido textual.

Paso 4 – Cambiar estilos con JavaScript

Agregar en script.js las siguientes instrucciones para modificar la apariencia visual del párrafo dinámico:

```
mensajeDinamico.style.backgroundColor = "#e8f5e9";  
mensajeDinamico.style.color = "#18529D";  
mensajeDinamico.style.padding = "12px";  
mensajeDinamico.style.borderLeft = "5px solid #28AD56";  
mensajeDinamico.style.fontWeight = "bold";
```



Paso 5 – Cambiar atributos de elementos HTML

A continuación, se modificarán atributos existentes de la imagen principal en la sección de presentación y múltiples enlaces de navegación.

Agregar en script.js:

```
//Cambiar atributo de Contacto de menú de navegación
```

```
const enlaceContacto = document.querySelector('nav a[href="#contacto"]');  
enlaceContacto.setAttribute("title", "Ir a la sección de contacto y reporte  
de incidente");
```

Paso 6 – Crear dinámicamente una sección de acciones rápidas

En este paso se generará una nueva sección llamada “Acciones rápidas” desde JavaScript. Agregar en script.js:

```
//Sección acciones rápidas  
const accionesRapidas = document.createElement("section");  
accionesRapidas.id = "acciones-rapidas";  
accionesRapidas.innerHTML = `  
  <h2>Acciones rápidas</h2>  
  <p>Utiliza los siguientes botones para interactuar con el portal.</p>  
  <button type="button" id="btn-resaltar-avisos">Resaltar avisos</button>  
  <button type="button" id="btn-ocultar-primer-aviso">Eliminar primer  
aviso</button>  
  <button type="button" id="btn-restaurar-mensaje">Restaurar  
mensaje</button>  
`;  
`;
```

Paso 7 – Insertar la nueva sección en el documento

Agregar inmediatamente después del código anterior:

```
//Insertar la sección "Acciones rápidas" en el documento  
const seccionServicios = document.getElementById("servicios");  
seccionServicios.insertAdjacentElement("beforebegin", accionesRapidas);
```

La instrucción insertAdjacentElement("beforebegin", ...) inserta la nueva sección antes del elemento #servicios

Paso 8 – Asociar eventos a los botones creados

Agrega en script.js el siguiente bloque para incorporar acciones, a través de botones, sobre diversos elementos del portal:



```
//8. Eventos de "Acciones rápidas"
document.getElementById("btn-resaltar-avisos").addEventListener("click",
function () {
    const avisos = document.querySelectorAll(".aviso");

    avisos.forEach(function (aviso) {
        aviso.style.backgroundColor = "#fff8e1";
        aviso.style.borderLeft = "6px solid #f9a825";
    });
});

document.getElementById("btn-restaurar-mensaje").addEventListener("click",
function () {
    mensajeDinamico.textContent = "Bienvenido al Portal Institucional del
Centro de Cómputo.";
    mensajeDinamico.style.backgroundColor = "#e8f5e9";
    mensajeDinamico.style.color = "#18529D";
});

document.getElementById("btn-ocultar-primer-
aviso").addEventListener("click", function () {
    const primerAviso = document.querySelector(".aviso");

    if (primerAviso) {
        primerAviso.remove();
        mensajeDinamico.textContent = "El primer aviso fue eliminado
correctamente.";
    } else {
        mensajeDinamico.textContent = "Ya no existen avisos para eliminar.";
    }
});
```

Paso 9 – Incorporar un carrusel de imágenes

Para hacer más atractiva la presentación del portal, se agregará en index.html, inmediatamente después de que cierre el elemento `<section aria-labelledby="titulo-portal">`, el siguiente bloque que corresponde a un carrusel de imágenes:

```
<section id="galeria" aria-labelledby="galeria-titulo">
    <h2 id="galeria-titulo">Galería del Centro de Cómputo</h2>
    
    <p id="descripcion-carrusel">Vista general del laboratorio.</p>
    <p>
        <button type="button" id="btn-anterior">Anterior</button>
        <button type="button" id="btn-siguiente">Siguiente</button>
    </p>
</section>
```



```
</p>  
</section>
```

Agregar en script.js:

```
//Carrusel  
const imagenesCarrusel = [  
  {  
    src: "img/portal-institucional.jpg",  
    alt: "Vista general del laboratorio de cómputo",  
    descripcion: "Vista general del laboratorio."  
  },  
  {  
    src: "img/centro-computo-2.jpg",  
    alt: "Equipo de cómputo en el Centro de Cómputo",  
    descripcion: "Equipos disponibles para actividades académicas."  
  },  
  {  
    src: "img/centro-computo-3.jpg",  
    alt: "Área de trabajo del Centro de Cómputo",  
    descripcion: "Espacios de trabajo para estudiantes y personal."  
  }  
];  
  
let indiceActual = 0;  
  
const imagenCarrusel = document.getElementById("imagen-carrusel");  
const descripcionCarrusel = document.getElementById("descripcion-carrusel");  
  
function mostrarImagen(indice) {  
  imagenCarrusel.src = imagenesCarrusel[indice].src;  
  imagenCarrusel.alt = imagenesCarrusel[indice].alt;  
  descripcionCarrusel.textContent = imagenesCarrusel[indice].descripcion;  
}  
  
document.getElementById("btn-siguiente").addEventListener("click", function  
( ) {  
  indiceActual++;  
  
  if (indiceActual >= imagenesCarrusel.length) {  
    indiceActual = 0;  
  }  
  
  mostrarImagen(indiceActual);  
});
```



```
document.getElementById("btn-anterior").addEventListener("click", function  
( ) {  
    indiceActual--;  
  
    if (indiceActual < 0) {  
        indiceActual = imagenesCarrusel.length - 1;  
    }  
  
    mostrarImagen(indiceActual);  
});
```

En este código del carrusel se está empleando: un arreglo de objetos, funciones, cambio de atributos, cambio de texto y eventos de clic.

Finalmente eliminar el elemento `<figure>` de la página HTML, con el objetivo no duplicar este elemento con el carrusel:

```
<figure>  
      
    <figcaption>Laboratorio de cómputo – recursos tecnológicos para  
actividades académicas.</figcaption>  
</figure>
```

Paso 10 – Aplicar funciones flecha en una mejora visual del menú portal

Además de las funciones tradicionales, JavaScript permite utilizar funciones flecha, también conocidas como arrow functions, las cuales ofrecen una sintaxis más compacta. Para practicar su uso, agregar en script.js el siguiente bloque:

```
//mouseover mouseout sobre enlaces de menú  
const enlacesMenu = document.querySelectorAll("nav a");  
  
enlacesMenu.forEach((enlace) => {  
    enlace.addEventListener("mouseover", () => {  
        enlace.style.transform = "scale(1.05)";  
    });  
  
    enlace.addEventListener("mouseout", () => {  
        enlace.style.transform = "scale(1)";  
    });  
});
```

Así, para cada enlace del menú se asocian dos eventos:



- mouseover, que agranda ligeramente el enlace,
- mouseout, que lo regresa a su tamaño original.

Paso 11 – Agregar una zona de carga de archivos con drag and drop Validar el formulario de contacto

La página contiene un formulario con campos de nombre, correo, rol, tipo y descripción. Agregar en index.html, justo después del formulario, el siguiente párrafo:

```
<p id="mensaje-formulario"></p>
```

Quedando en la siguiente ubicación:

```
...<p>
    <button type="submit">Enviar</button>
    <button type="reset">Limpiar</button>
</p>
</fieldset>
</form>
<p id="mensaje-formulario"></p>
</section>
```

Cambiar el elemento formulario:

```
<form action="#" method="post" aria-label="Formulario de contacto">
```

Agregando `novalidate`, de la siguiente forma:

```
<form action="#" method="post" aria-label="Formulario de contacto"
novalidate>
```

De esta forma la validación la hará el script que estás creando y no HTML5.

Para agregar una zona de carga de archivos con drag and drop, en el formulario de la página HTML, **antes** de:

```
<p>
    <button type="submit">Enviar</button>
    <button type="reset">Limpiar</button>
</p>
```

Agrega:

```
<section id="evidencia" aria-labelledby="evidencia-titulo">
    <h3 id="evidencia-titulo">Carga de evidencia</h3>
    <p>Adjunta un archivo como evidencia del incidente.</p>
```



```
<div id="zona-arrastre" tabindex="0">
  Arrastra aquí un archivo o haz clic para seleccionarlo.
</div>

<input type="file" id="archivo-evidencia" name="archivo-evidencia"
hidden>

<p id="archivo-seleccionado">Ningún archivo seleccionado.</p>
</section>
```

En la hoja de estilos.css agrega lo siguiente:

```
/*Estilos para Drag and Drop*/
#zona-arrastre {
  border: 2px dashed #18529D;
  padding: 20px;
  text-align: center;
  background-color: #f8fbff;
  border-radius: 6px;
  cursor: pointer;
  margin-top: 10px;
}

#zona-arrastre.activa {
  background-color: #e3f2fd;
  border-color: #28AD56;
}

#archivo-seleccionado {
  margin-top: 10px;
  font-weight: bold;
}

#mensaje-formulario {
  margin-top: 10px;
  font-weight: bold;
}
```

En el archivo script.js agrega lo siguiente para añadir funcionalidad a la zona drag and drop:

```
//Zona Drag and Drop para carga de archivos evidencia
const formulario = document.querySelector("form");
const campoNombre = document.getElementById("nombre");
const campoCorreo = document.getElementById("correo");
const campoRol = document.getElementById("rol");
```



```
const campoTipo = document.getElementById("tipo");
const campoDescripcion = document.getElementById("descripcion");
const inputArchivo = document.getElementById("archivo-evidencia");
const zonaArrastre = document.getElementById("zona-arrastre");
const archivoSeleccionado = document.getElementById("archivo-seleccionado");
const mensajeFormulario = document.getElementById("mensaje-formulario");

/* Variable para conservar el archivo actual, ya sea seleccionado o
arrastrado */
let archivoActual = null;

/* Función para mostrar el nombre del archivo seleccionado */
const mostrarArchivo = (archivo) => {
    archivoSeleccionado.textContent = "Archivo seleccionado: " + archivo.name;
    archivoSeleccionado.style.color = "#18529D";
};

/* Abrir selector de archivos al hacer clic */
zonaArrastre.addEventListener("click", () => {
    inputArchivo.click();
});

/* También permitir activarlo con teclado */
zonaArrastre.addEventListener("keydown", (event) => {
    if (event.key === "Enter" || event.key === " ") {
        event.preventDefault();
        inputArchivo.click();
    }
});

/* Cuando se selecciona un archivo desde el explorador */
inputArchivo.addEventListener("change", () => {
    try {
        if (inputArchivo.files.length === 0) {
            throw new Error("No se seleccionó ningún archivo.");
        }

        archivoActual = inputArchivo.files[0];
        mostrarArchivo(archivoActual);
    } catch (error) {
        archivoActual = null;
        archivoSeleccionado.textContent = "Error: " + error.message;
        archivoSeleccionado.style.color = "red";
    }
});
```



```
/* Permitir arrastrar archivo sobre la zona */
zonaArrastre.addEventListener("dragover", (event) => {
    event.preventDefault();
    zonaArrastre.classList.add("activa");
});

/* Quitar estilo visual al salir de la zona */
zonaArrastre.addEventListener("dragleave", () => {
    zonaArrastre.classList.remove("activa");
});

/* Soltar archivo en la zona */
zonaArrastre.addEventListener("drop", (event) => {
    event.preventDefault();
    zonaArrastre.classList.remove("activa");

    try {
        const archivos = event.dataTransfer.files;

        if (archivos.length === 0) {
            throw new Error("No se arrastró ningún archivo.");
        }

        archivoActual = archivos[0];
        mostrarArchivo(archivoActual);
    } catch (error) {
        archivoActual = null;
        archivoSeleccionado.textContent = "Error: " + error.message;
        archivoSeleccionado.style.color = "red";
    }
});

/* Limpiar mensaje visual al reiniciar el formulario */
formulario.addEventListener("reset", () => {
    archivoActual = null;
    archivoSeleccionado.textContent = "Ningún archivo seleccionado.";
    archivoSeleccionado.style.color = "";
    mensajeFormulario.textContent = "";
});
```

Y a continuación añadir el siguiente código (en script.js) para validar el formulario:

```
//Validación de formulario

formulario.addEventListener("submit", function (event) {
    event.preventDefault();
```



```
if (campoNombre.value.trim() === "") {  
    mensajeFormulario.textContent = "El nombre completo es obligatorio.";   
    mensajeFormulario.style.color = "red";  
    campoNombre.focus();  
    return;  
}  
  
if (!campoCorreo.value.includes("@")) {  
    mensajeFormulario.textContent = "El correo institucional no es válido.";   
    mensajeFormulario.style.color = "red";  
    campoCorreo.focus();  
    return;  
}  
  
if (campoRol.value === "") {  
    mensajeFormulario.textContent = "Debes seleccionar un rol.";   
    mensajeFormulario.style.color = "red";  
    campoRol.focus();  
    return;  
}  
  
if (campoTipo.value === "") {  
    mensajeFormulario.textContent = "Debes seleccionar un tipo de   
solicitud.";   
    mensajeFormulario.style.color = "red";  
    campoTipo.focus();  
    return;  
}  
  
if (campoDescripcion.value.trim().length < 10) {  
    mensajeFormulario.textContent = "La descripción debe contener al menos   
10 caracteres.";   
    mensajeFormulario.style.color = "red";  
    campoDescripcion.focus();  
    return;  
}  
  
if (!archivoActual) {  
    mensajeFormulario.textContent = "Debes adjuntar un archivo de   
evidencia.";   
    mensajeFormulario.style.color = "red";  
    zonaArrastre.focus();  
    return;  
}
```



```
mensajeFormulario.textContent = "Formulario validado correctamente. Datos  
listos para enviarse.";
mensajeFormulario.style.color = "green";
});
```

Paso 12- Incorporar una mejora: modo nocturno

Para ofrecer una mejor experiencia al usuario, se agregará un botón que alterne un modo nocturno del portal. Agregar en script.js:

```
const btnModo = document.createElement("button");
btnModo.type = "button";
btnModo.id = "btn-modo";
btnModo.textContent = "Activar modo nocturno";

document.querySelector("header").insertAdjacentElement("beforeend",
btnModo);

btnModo.addEventListener("click", function () {
    document.body.classList.toggle("modo-nocturno");

    if (document.body.classList.contains("modo-nocturno")) {
        btnModo.textContent = "Desactivar modo nocturno";
    } else {
        btnModo.textContent = "Activar modo nocturno";
    }
});
```

Agregar en css/estilos.css:

```
/*Modo nocturno*/
.modo-nocturno {
    background-color: #1e1e1e;
    color: #f5f5f5;
}

.modo-nocturno main section,
.modo-nocturno aside {
    background-color: #2c2c2c;
    color: #f5f5f5;
}

.modo-nocturno table,
.modo-nocturno th,
.modo-nocturno td {
    color: #f5f5f5;
}
```



Paso 13 – Verificar el funcionamiento general de la página

Abrir la página en el navegador y comprobar lo siguiente:

- a) El mensaje dinámico cambia al cargar la página.
- b) Los estilos del mensaje se actualizan correctamente.
- c) La sección “Acciones rápidas” aparece antes de la sección de servicios.
- d) Los botones de acciones rápidas funcionan correctamente.
- e) El primer aviso puede eliminarse.
- f) El carrusel cambia de imagen al pulsar los botones correspondientes.
- g) Los enlaces del menú responden visualmente al pasar el cursor.
- h) El formulario valida los datos y muestra mensajes adecuados.
- i) La zona de drag and drop responde al clic, al arrastre y al soltado del archivo.
- k) El modo nocturno cambia la apariencia general del portal.

RESULTADOS ENTREGABLES:

1. Un archivo .zip con la carpeta del proyecto.
2. Los siguientes archivos correctamente organizados:
 - index.html
 - css/estilos.css
 - js/script.js
 - carpeta img
3. La página debe funcionar correctamente en navegador.
4. El código debe conservar una estructura ordenada e indentación adecuada.
5. Capturas de pantalla que evidencien:
 - Cambio de mensaje dinámico,
 - Sección de acciones rápidas insertada en la página,
 - Eliminación de un aviso,
 - Funcionamiento del carrusel,
 - Validación del formulario,
 - Zona de carga de archivos con drag and drop,
 - Modo nocturno activado.